

Reporte de Vulnerabilidad

Archivo: ProductController.cs

Código Analizado:

```
?using FermaOrders.API.Controllers.Response;
using FermaOrders.Application.Dto.Components.Product;
using FermaOrders.Application.Interface.Components;
using FermaOrders.Application.Service.Components;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Net;

namespace FermaOrders.API.Controllers.Application
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductController(IProductService productService)
        {
            _productService = productService;
        }

        // Crear un asiento
        [Authorize(Roles = "Admin")]
        [HttpPost]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> CreateProductAsync([FromBody] ProductCreateDto
productDto)
        {
            var response = new RespuestaAPI();
            try
            {
                if (productDto == null)
                {
                    return BadRequest("Invalid Product data.");
                }

                var product = await _productService.CreateProductAsync(productDto);
                response.Result = product;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        // Obtener un asiento por ID
        [Authorize(Roles = "Admin")]
        [HttpGet("{id}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetProductByIdAsync(int id)
        {
            var response = new RespuestaAPI();

            try
            {
                var product = await _productService.GetProductByIdAsync(id);
```

```

        if (product == null)
        {
            return BadRequest("Invalid Product data.");
        }

        response.Result = product;
        response.IsSuccess = true;
        response.StatusCode = HttpStatusCode.OK;
        return Ok(response);
    }
    catch (Exception ex)
    {
        response.StatusCode = HttpStatusCode.InternalServerError;
        response.IsSuccess = false;
        response.ErrorMessages.Add($"Error: {ex.Message}");
        return StatusCode(500, response);
    }
}

[Authorize(Roles = "Admin")]
[HttpPut("{id}")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult> UpdateProductAsync(int id, [FromBody]
ProductUpdateDto productDto)
{
    var response = new RespuestaAPI();

    try
    {
        if (id != productDto.Id)
        {
            return BadRequest("ID mismatch.");
        }
        await _productService.UpdateProductDto(productDto);

        response.IsSuccess = true;
        response.StatusCode = HttpStatusCode.OK;
        response.Result = new { message = "Product updated successfully." };
        return Ok(response);
    }
    catch (Exception ex)
    {
        response.StatusCode = HttpStatusCode.InternalServerError;
        response.IsSuccess = false;
        response.ErrorMessages.Add($"Error: {ex.Message}");
        return StatusCode(500, response);
    }
}

// Eliminar un asiento
[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult> DeleteProductAsync(int id)
{
    var response = new RespuestaAPI();
    try
    {
        await _productService.DeleteProductAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException ex)
    {
        response.StatusCode = HttpStatusCode.NotFound;
        response.IsSuccess = false;
        response.ErrorMessages.Add($"Error: {ex.Message}");
        return NotFound(response);
    }
    catch (Exception ex)
    {
        response.StatusCode = HttpStatusCode.InternalServerError;
        response.IsSuccess = false;
        response.ErrorMessages.Add($"Error: {ex.Message}");
        return StatusCode(500, response);
    }
}

```

```

    }
}

[Authorize(Roles = "Admin")]
[HttpGet("products")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status403Forbidden)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> GetProductList()
{
    var response = new RespuestaAPI();

    try
    {
        var products = await _productService.ListProduct();

        response.Result = products;
        response.IsSuccess = true;
        response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.OK;

        return Ok(response);
    }
    catch (Exception ex)
    {
        response.IsSuccess = false;
        response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError;
        response.ErrorMessages.Add($"Error: {ex.Message}");

        return StatusCode(500, response);
    }

    // Redundante, pero garantiza que todas las rutas retornan algo
    // No debería alcanzarse nunca
    return StatusCode(500, new RespuestaAPI
    {
        IsSuccess = false,
        StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError,
        ErrorMessages = new List<string> { "Ocurrió un error inesperado." }
    });
}
}
}

```

Análisis: ``html

Análisis de Vulnerabilidades y Métricas de Calidad del Código

Resumen

El código presenta un manejo básico de excepciones y una estructura CRUD para productos. La autenticación y autorización se implementan mediante roles. Sin embargo, se identifican áreas de mejora en cuanto a manejo de errores, validación de datos y consistencia.

Vulnerabilidades de Seguridad

- Vulnerabilidad:** Manejo Insuficiente de Errores en `GetProductByIdAsync`
Línea Aproximada: Alrededor de la línea 54: `if (product == null)`
Descripción: Si no se encuentra el producto, se devuelve un BadRequest con "Invalid Product data.". Esto puede ser confuso para el cliente. Además, `BadRequest` es para errores del *cliente* (ej: data mal formada), no errores del *servidor* (ej: no se encuentra el dato en la DB). Es mejor un `NotFound`
Mitigación: Retornar un `NotFound` con un mensaje más informativo ("Product not found"). Esto comunica mejor el problema al cliente y sigue las convenciones HTTP.
Métrica Mejorada: Claridad del código, disminuyendo la confusión en el manejo de errores.
- Vulnerabilidad:** Falta de validación de entrada robusta en `CreateProductAsync` y `UpdateProductAsync`.
Línea Aproximada: Alrededor de la línea 34 y 78.
Descripción: Si bien se verifica que `productDto` no sea nulo, no se valida el contenido del DTO. Esto puede llevar a errores inesperados o a la persistencia de datos inválidos.

Mitigación: Implementar validaciones más completas en ``ProductCreateDto`` y ``ProductUpdateDto`` utilizando Data Annotations u otras bibliotecas de validación. Validar, por ejemplo, que los campos obligatorios no estén vacíos, que los rangos de valores sean correctos, etc.

Métrica Mejorada: Robustez del código, previniendo errores causados por datos inválidos.

- **Vulnerabilidad:** Exposición excesiva de detalles de la excepción.

Línea Aproximada: Bloques ``catch`` en todos los métodos.

Descripción: En caso de error, se retorna el mensaje completo de la excepción al cliente. Esto puede exponer información sensible sobre la implementación interna.

Mitigación: Registrar la excepción para análisis interno, pero retornar al cliente un mensaje genérico de error (ej. "Ocurrió un error al procesar la solicitud"). Usar un código de error específico (ej. 503 Service Unavailable) puede ser mas adecuado para algunos tipos de errores.

Métrica Mejorada: Seguridad, previniendo la fuga de información sensible.

- **Vulnerabilidad:** Código redundante en ``GetProductList``

Línea Aproximada: 132 - 137

Descripción: La función ``GetProductList`` tiene un bloque de código después del ``try-catch``, que es innecesario porque el bloque `try-catch` siempre retorna un valor.

Mitigación: Eliminar el código redundante para evitar confusiones y reducir la posibilidad de errores.

Métrica Mejorada: Legibilidad y mantenibilidad del código

Métricas de Calidad del Código

- **Complejidad:** La complejidad ciclomática es baja en general. Sin embargo, el manejo repetitivo de excepciones en cada método podría simplificarse con un middleware global de manejo de excepciones.
Mejora: Implementar un middleware para el manejo centralizado de excepciones.
- **Duplicación:** Existe duplicación en el manejo de excepciones y la construcción de la respuesta ``RespuestaAPI``.
Mejora: Crear un método auxiliar para construir la respuesta ``RespuestaAPI`` y centralizar el manejo de excepciones en un middleware.
- **Legibilidad:** El código es relativamente legible, pero la consistencia en los nombres de las variables y el uso de comentarios podrían mejorarse.
Mejora: Adoptar una convención de nombres consistente y agregar comentarios relevantes para aclarar la lógica del código.
- **Acoplamiento:** El controlador depende directamente de la interfaz ``IProductService``. Esto es adecuado. El acoplamiento podría reducirse aún más utilizando inyección de dependencias y patrones de diseño.
Mejora: Revisar la necesidad de abstracciones adicionales o patrones de diseño para reducir el acoplamiento entre componentes.

...